CONTINUOUS ASSESSMENT 3 — MINI PROJECT 2 REPORT

Course Code: CSB312 | Generative AI Foundations

Project Title: AtlasForge — AI Research & Knowledge Navigator (Option 5)

Student Name: Ashmit Thakur	**Registration / Roll No.:** 12408293
Selected Concept: Option 5 — AtlasForge (Idea 1)	**Deployed Frontend URL:** https://atlasforge-sooty.vercel.app

1. PROJECT OVERVIEW

1.1 Selected Automation Idea

Option 5 (AtlasForge — AI Research & Knowledge Navigator), Automation Idea 1: *"Convert a research paper or uploaded document into structured claims, methods, limitations, and keywords."*

1.2 Problem Statement

Students and researchers read many academic papers, but critical information—such as key claims, methodologies, author-stated limitations, and keywords—remains buried inside long, unstructured PDFs. Manually extracting and organizing this data is slow and inconsistent, while standard chatbot summaries generate free-form text that cannot be validated, stored in relational columns, or queried reliably.

1.3 Project Objective

Build and deploy a full-stack Generative AI application that: (1) accepts a research paper as a PDF upload (up to 4 MB) or pasted text (minimum 200 characters); (2) uses schema-guided prompting with Google Gemini so the AI returns data in a fixed JSON structure; (3) validates the result with Zod and never silently invents missing values (returning `null` or `[]` and listing absent fields in `missing_fields`); (4) shows a preview requiring user confirmation; and (5) creates a record in Supabase PostgreSQL and displays the saved record confirmation on the frontend.

2. END-TO-END WORKFLOW

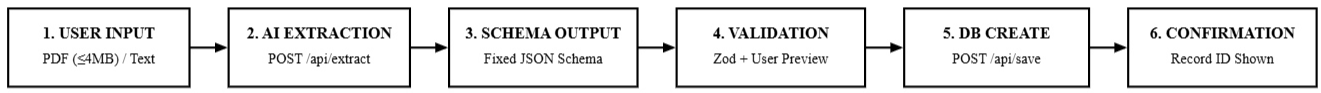


Figure 1: Required 6-Stage End-to-End Schema-Guided Pipeline

3. SYSTEM ARCHITECTURE

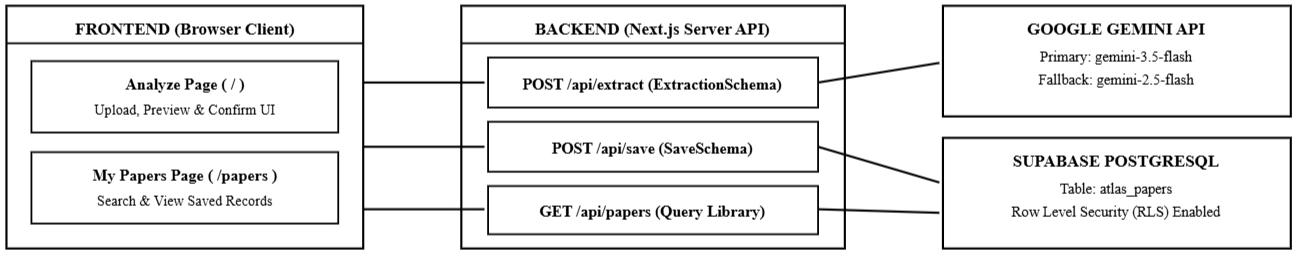


Figure 2: Full-Stack System Architecture (Next.js 16, Gemini API, Zod, and Supabase PostgreSQL)

4. TECHNOLOGY STACK

Layer	Technology	Role in Application
Frontend	Next.js 16 (App Router), React 19, Tailwind CSS 4	Input interface, validation badges, preview confirmation screen, and searchable paper library.
Backend API	Next.js Server Route Handlers	Server-only endpoints (`/api/extract`, `/api/save`, `/api/papers`) protecting secret keys.
AI Component	Google Gemini (`@google/genai`)	Structured output with JSON Schema, PDF support, and automatic retry + fallback model.
Validation	Zod	Two-level server validation: `ExtractionSchema` (after AI call) and `SaveSchema` (before DB insert).
Database & Host	Supabase PostgreSQL & Vercel	Managed PostgreSQL table (`atlas_papers` with RLS enabled) and cloud deployment from GitHub.

5. SCHEMA DEFINITION & MISSING-FIELD HANDLING (LIB/SCHEMA.JS)

The same 10-field structure is used in two places: as a JSON Schema sent to Gemini (`EXTRACTION_JSON_SCHEMA`) to force the output shape, and as Zod schemas on the server (`ExtractionSchema` and `SaveSchema`) to validate that output. Missing values are never guessed: absent values return `null` or `[]` and are listed in `missing_fields`.

Field Name	Data Type	Nullable	Validation Rule & Missing-Value Behavior
`is_research_paper`	`boolean`	No	`false` if input is not academic content (e.g., recipe or story); rejected with HTTP 422.
`title`	`string | null`	Yes	Exact paper title. Required to save (minimum 3 characters in `SaveSchema`).
`authors`	`string[]`	No	Author names in order; empty array `[]` if not found.
`publication_year`	`integer | null`	Yes	Only if explicitly stated (1900 to 2100); otherwise `null`.
`research_question`	`string | null`	Yes	Main research question or objective; `null` if not found.
`claims`	`string[]`	No	Maximum 5 core findings; at least 1 claim is required in `SaveSchema` to save.
`methods`	`string[]`	No	Maximum 5 methods or experimental setups; `[]` if not found.
`limitations`	`string[]`	No	Maximum 5 limitations only if stated by the authors; `[]` if absent.
`keywords`	`string[]`	No	3 to 8 lowercase keywords.
`missing_fields`	`string[]`	No	Names of fields above that could not be found in the provided document.

6. PROMPT DESIGN & MULTI-LEVEL VALIDATION STRATEGY

6.1 System Instruction Rules & Generation Settings (lib/prompt.js)

- **Source-Only Extraction:** Extract information only from the provided document; never invent, guess, or use outside knowledge.
- **Honest Missing-Value Handling:** Use `null` or `[]` when a value is absent and list every missing field name in `missing_fields`.
- **Non-Research Detection:** If the input is not research content, set `is_research_paper` to `false`.
- **Generation Settings:** `responseMimeType: "application/json"`, `responseJsonSchema: EXTRACTION_JSON_SCHEMA`, and `temperature: 0.1`.

6.2 Three-Level Validation & Retry/Fallback Strategy (lib/gemini.js)

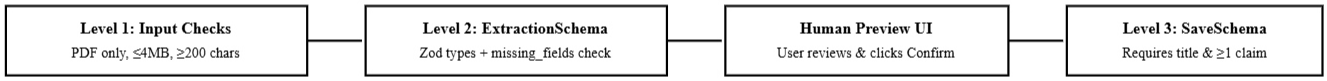


Figure 3: Three-Level Validation Pipeline Before Database Insert

To handle temporary AI overload, `lib/gemini.js` tries the main model (`gemini-3.5-flash`, up to 2 attempts) and automatically falls back to `gemini-2.5-flash` (up to 2 attempts) on HTTP 503, 429, or 500 errors.

7. SAMPLE STRUCTURED AI OUTPUT

Real output from uploading the *OpenTac* PDF. Note that `publication_year` is `null` and `limitations` is `[]` because they were not explicitly stated, and both are listed in `missing_fields` instead of being guessed:

```
{
  "is_research_paper": true,
  "title": "OpenTac: An Open Ecosystem for Customizable, Scalable, and Transferable 3D Tactile Sensing",
  "authors": ["Ariel Slepyan", "Geeta Chandra Raju Bethala", "Jiafeng Gu"],
  "publication_year": null,
  "research_question": "How can an open full-stack ecosystem enable customizable, scalable tactile skins
for robotics?",
  "claims": [
    "OpenTac-CAD automatically generates sensor fabrication files from 2D outlines, 3D meshes, or text
prompts.",
    "OpenTac-Xampling enables kilohertz-rate partial readouts, 19.2x faster than full raster scanning."
  ],
  "methods": ["Piezoresistive tactile skin with elastomeric bump overlays", "OpenTac-Sim built on
MuJoCo"],
  "limitations": [],
  "keywords": ["tactile sensing", "robotics", "sensor design"],
  "missing_fields": ["publication_year", "limitations"]
}
```

8. DATABASE DESIGN & CREATE OPERATION (SUPABASE POSTGRESQL)

8.1 SQL Table Definition (atlas_papers)

```
create table atlas_papers (
  id                bigint generated always as identity primary key,
  title             text not null,
  authors           text[] not null default '{}',
  publication_year  int,
  research_question text,
  claims            text[] not null,
  methods           text[] not null default '{}',
  limitations       text[] not null default '{}',
  keywords          text[] not null default '{}',
  missing_fields    text[] not null default '{}',
  source_type       text not null check (source_type in ('pdf', 'text')),
  source_name       text,
  created_at        timestamptz not null default now()
);
alter table atlas_papers enable row level security;
```

8.2 Explanation of Database Create & Read Operations

- **Separate Extract and Save Calls:** `POST /api/extract` extracts and validates the paper without saving anything. Only when the user clicks **"Confirm & Save"** on the preview screen does the browser call `POST /api/save`.
- **Create Operation (`POST /api/save`):** Builds a record using only allowed keys, validates it on the server with `SaveSchema` (requiring a valid `title` and at least one `claim`), and executes `insert(...).select().single()`. The newly created row (with its generated `id` and `created_at`) is returned with HTTP `201` and displayed on the frontend as *"Record #ID created at DATE TIME"*.
- **Read Operation (`GET /api/papers`):** Returns the latest 50 saved papers (newest first) with optional title search (`?q=`).
- **Security & Safeguards:** Secret keys (`GEMINI_API_KEY`, `SUPABASE_SERVICE_ROLE_KEY`) stay strictly on the server. Row Level Security (RLS) is enabled on `atlas_papers` with no public policies, blocking direct public access.

9. TESTING & VALIDATION EVIDENCE

9.1 Automated Verification Scripts

Test Script	Command	Scope	Result
Database Connection	`node --env-file=.env.local scripts/test-db.mjs`	DB connection OK	**PASS**
Schema Validation	`node scripts/test-schema.mjs`	All schema tests	**PASS**
Extraction API	`node scripts/test-extract.mjs`	All extract tests	**PASS**
Save & List API	`node scripts/test-save.mjs`	All save tests	**PASS**
Production Build	`npm run build`	Compiled cleanly	**PASS**

9.2 Manual Test Cases (Successful Flow, Missing Fields & Error Cases)

#	Case	Input	Expected & Observed Result	Status
1	Successful flow	Real research paper PDF	Preview shown, then saved to Supabase with Record ID	**201 PASS**
2	Missing fields	PDF without explicit year/limitations	Shown as "Not found" and listed in `missing_fields`	**200 PASS**
3	Empty input	Nothing selected	400: *"Please upload a PDF or paste some text."*	**400 PASS**
4	Not a research paper	Recipe text	422: *"This does not look like a research paper."*	**422 PASS**
5	Wrong file type	`.jpg` file	400: *"Only PDF files are supported."*	**400 PASS**
6	File too large	9.7 MB PDF (> 4 MB limit)	400: *"PDF is too large. Maximum size is 4 MB."*	**400 PASS**
7	Incomplete save	Title `null`, no claims	422: *"Cannot save: required information is missing."*	**422 PASS**
8	AI overloaded	Gemini 503 error	Automatic retry + fallback to `gemini-2.5-flash`	**200 PASS**

10. INTERFACE, DATABASE & TESTING SCREENSHOTS

No.	Screen	Repository Image Path	Verification Description
1	Analyze page	`docs/screenshots/01-home.png`	PDF upload (max 4 MB) and text paste input interface.
2	Preview with schema valid badge & missing fields	`docs/screenshots/02-preview.png`	Extracted fields preview, schema valid badge, and missing fields list.
3	Structured JSON output	`docs/screenshots/03-json.png`	Raw JSON matching `EXTRACTION_JSON_SCHEMA` and model used.
4	Saved to database confirmation	`docs/screenshots/04-saved.png`	Frontend confirmation showing created Record ID and timestamp.
5	My Papers list	`docs/screenshots/05-my-papers.png`	Searchable list of saved papers with expandable details.
6	Supabase table with saved rows	`docs/screenshots/06-supabase.png`	Supabase PostgreSQL `atlas_papers` table showing inserted records.
7	Error: not a research paper	`docs/screenshots/07-error-not-research.png`	422 error message when non-research text is submitted.
8	Error: file too large	`docs/screenshots/08-error-too-large.png`	400 error message when a PDF larger than 4 MB is selected.

11. KNOWN LIMITATIONS & FUTURE IMPROVEMENTS

11.1 Known Limitations

- **4 MB PDF Limit:** Vercel serverless functions reject request bodies above about 4.5 MB.
- **Scanned (Image-Only) PDFs:** Scanned PDFs may extract poorly compared to text-based PDFs.
- **Slight Output Variation:** AI output can vary slightly between runs, which is why human review before saving is required.
- **Free-Tier Limits:** Gemini free-tier rate limits apply, and Supabase free projects pause after 7 days of inactivity.
- **Create & Read Only:** Only create and read operations are implemented; editing and deleting records are not in the UI.
- **Search & Shared List:** Search is by title only, and there are no user accounts (all saved papers share one list).

11.2 Future Improvements

- Let users edit extracted fields directly in the preview screen before saving.
- Support updating and deleting records with a preserved change history.
- Search by keywords, authors, and claims, plus filter by publication year.
- Compare multiple papers and build a topic-wise evidence map.
- Add user authentication for private libraries, direct storage upload for large PDFs, OCR for scanned papers, and CSV/BibTeX export.

12. DEPLOYMENT INFORMATION (MANDATORY)

Deployed Frontend URL	**https://atlasforge-sooty.vercel.app**
Deployment Architecture	Code pushed to GitHub (`.env.local` ignored) and deployed on **Vercel** with encrypted environment variables (`GEMINI_API_KEY`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`).

13. CONCLUSION

AtlasForge demonstrates a complete, deployed Generative AI pipeline: unstructured research papers are converted into a fixed, validated schema by Gemini, reviewed by a human, and stored as structured records in a Supabase PostgreSQL database. Schema-guided prompting, multi-level Zod validation, honest handling of missing fields, resilient AI calls with model fallback, and server-side security together make the output reliable enough to store and reuse.